Verex: Verified Execution

Draft whitepaper · Oct 6, 2026 · @MOVE-ENG

Abstract

We describe Verified Execution (Verex), a platform for running code in Rust and similar languages whose failure is unacceptable, built so that no single component has to be trusted. Its foundation was built for the Move language and has run in production for years; Verex extends it to Rust and similar languages with an ownership and borrowing discipline. Programs are checked by a formal verifier before deployment, re-verified as bytecode at load time, and optionally checked again at runtime by an independent implementation of the same safety rules. Execution is deterministic, metered and isolated. Where interpreted execution is too slow, verified Rust can be compiled to native code, and the VM serves for simulation, shadow execution and differential testing.

The Move foundation has run the Aptos blockchain for several years, in an adversarial setting where assets worth billions of dollars are at stake and anyone can submit hand-crafted bytecode. Its runtime checks have stopped real exploits that passed the static verifier, and its new virtual machine, MonoVM, executes 30-50 times faster than its predecessor.

Verex extends the platform to sequential, deterministic Rust, including recursive data structures, interior mutability, unsafe raw pointers and floating point. Safe Rust shares Move's reference discipline, so its safety is checked on bytecode without making the Rust compiler part of the trusted base. Unsafe code runs under a checked memory model: every raw-pointer access can be checked at runtime, and the prover can show that no check fails. Misuse such as use-after-free becomes a defined, located abort instead of undefined behavior. Unsafe is supported by the platform to allow reuse of existing verified Rust crates.

The targets are high-assurance systems: code that is increasingly written by machines and reviewed by fewer people (agent-generated programs, financial logic and safety-critical control software) and existing Rust libraries whose unsafe internals nobody has fully audited.

1. The problem

Critical software is increasingly produced faster than people can review it. AI systems now write a growing share of production code, and agents generate and run programs on the fly. At the same time, AI-assisted auditing finds vulnerabilities faster, so any flaw that exists should be expected to be found. Supply chain attacks add a further threat: a compromised dependency, compiler or build tool can inject code that no one has reviewed.

Current assurance rests on a chain of trusted components: the developer, the reviewer, the compiler, the analysis tools and the runtime. Each is complex software, and each is a single point of failure for the guarantees built on it.

- **Memory-safe languages** remove a class of bugs, but trust the compiler, discard their guarantees at the binary boundary, and give no guarantees inside `unsafe` code, which most real systems contain somewhere.
- **Sandboxes** such as WebAssembly isolate code, but see only an untyped memory blob and cannot enforce ownership or resource rules. An authorization token, for example, should only come from an authorization step, but in untyped memory any code can forge its bytes.
- **Formal verifiers** prove properties of source code, but the proof says nothing about a bug in the compiler, the verifier or the runtime.

In settings where a single missed violation means loss of funds, a wrong control output or a forged authority, this is not enough. The question is how to run code that nobody fully trusts, with guarantees that survive a bug in any one component.

2. The product: Verex

Verex is for Rust and similar languages. Teams write ordinary Rust with their usual tools, and Verex verifies and runs it. The approach applies to any language with an ownership and borrowing discipline; Move, the language the platform was originally built for, is supported as well.

The prover, the verifier, the runtime checks and the VM each check the code on their own, and together they give one consistent guarantee.

Run code you do not trust and know what it can and cannot do.

For owners of high-assurance and safety-critical systems, Verex is a verification toolchain and a hardened runtime that work together. Before deployment, the toolchain proves the properties the system depends on: a control output stays within its envelope, a limit is never exceeded, a function never aborts. During execution, the runtime can check again that the code respects its types, resources and memory, independently of the compiler that produced it and the verifier that accepted it. These checks are optional; owners enable them where the extra assurance is worth their cost.

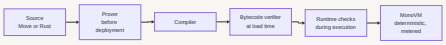
- **Mathematical proofs, not only tests.** Contracts and invariants are proved automatically in CI, with Lean proofs where kernel-checked evidence is needed. Specifications can be inferred and reviewed instead of written from scratch.
- **Independent redundancy.** The prover, the bytecode verifier and the runtime checks are independent mechanisms that cross-check each other. A bug in the compiler, the verifier or a dependency does not silently void the guarantees; with runtime checks enabled, the runtime catches the violation.
- **Containment of untrusted code.** Supplier components, third-party libraries, AI-generated code and supply-chain injections run isolated, and cannot forge authority or touch state beyond what their types permit.
- **Defined failure.** Memory errors in unsafe code, such as use-after-free or out-of-bounds access, are proved absent or, with runtime checks enabled, become located aborts instead of silent corruption, so the host can switch to a fallback.
- **Reproducible execution.** Execution is deterministic, including floating point, so every run can be replayed bit for bit for incident analysis, audit and certification evidence.
- **Bounded resources.** Every execution is metered, with explicit bounds on computation and memory.

Two deployment modes. Where its performance suffices, code runs on MonoVM with all layers in place. Where it does not, as in hard real-time control loops or high-throughput numeric kernels, the same verified Rust is compiled to native code. The proofs still hold for the native build, which then trusts the native compiler instead of the load-time and runtime checks. MonoVM remains in the loop for simulation, differential testing and pre-production, and as a redundant, independently implemented execution channel that runs alongside the native build and monitors it (shadow execution).

Adoption starts with one component where failure is most expensive (a control law, a limit check, a supplier plugin). It is compiled for the platform, its key properties are specified and proved, and it runs on the platform next to the existing system, which stays as it is, with runtime checks enabled as needed. The same technology already protects billions of dollars on the Aptos blockchain, where anyone can submit code (Section 4). Support for certification standards such as DO-178C is planned on this base (Section 10).

3. Design principle: no single trusted component

Every critical safety invariant (typing, resources, references and memory) can be established at least twice, by independent implementations at different stages. This is independent redundancy: a bug in one layer is caught by another. Runtime checks are optional: a deployment enables them where the extra assurance is worth their cost, either inline or asynchronously after commit.



The diagram shows the path from source to execution; each box after the compiler rechecks what earlier boxes established.

Layer	What it establishes	Trusts
Prover	Functional properties: pre- and postconditions, abort conditions, invariants	Specifications, SMT solver or Lean kernel; the compiler, to preserve proved properties in the bytecode
Bytecode verifier	Type, resource (ability) and reference safety of untrusted bytecode	Nothing from the compiler
Runtime checks	The same safety properties, on the executed path	Nothing from the verifier (separate code)
Execution engine	Determinism, resource bounds, metering, isolation	Its own implementation, tested by differential replay

Two properties of the language make this layering practical. First, the type system carries enough information to check safety on bytecode, so the compiler is never trusted. Second, references follow a strict aliasing discipline (one mutable reference or many immutable ones), which keeps both static and dynamic checking tractable and gives the prover an alias-free memory model.

Two cases have fewer layers. Functional properties are established by the prover alone, and the compiler is trusted to preserve them until compiler-correctness results close this gap (Section 10.1). Unsafe Rust is the one safety case the load-time verifier cannot cover, because raw-pointer accesses cannot be checked from types alone. For such code, the two layers are the runtime memory checks and the prover (Section 5.4); code without a proof relies on the runtime checks alone. Native deployments (Section 2) keep the proofs but trust the native compiler and have no load-time or runtime checks; differential testing against MonoVM is then the independent check.

Supply chain attacks. Injected code arrives through the same tools the platform declines to trust. When enabled, the runtime checks run on the bytecode that is actually executed and trust neither the compiler nor the verifier, so they hold even when the toolchain or a dependency is compromised. Injected code cannot forge an authorization token, duplicate an asset or reach a value it holds no reference to; an attempt that slips past the load-time verifier is stopped by the runtime type checks. Injected code can still do whatever its types permit: type and resource safety bound the damage, not every misbehavior.

4. The platform today

Every component of Section 3 is implemented and open source in the Aptos code base. The language, prover, bytecode verifier, and type and ability checks are hardened by years of production use on the Aptos blockchain, where anyone can submit hand-crafted bytecode and billions of dollars in assets are at stake. The runtime reference checker and MonoVM are newer and are validated by replaying historical mainnet transactions (Section 7).

- **Move:** a strongly typed language modeled after Rust with generics, algebraic data types, first-class functions [2], linear resource types and a borrow discipline close to Rust's; execution is deterministic.
- **Move Prover:** verifies contracts and global invariants with SMT solvers and Lean, fast enough to run in CI on every change [1, 5].
- **Bytecode verifier:** checks typing, abilities and reference safety at load time, independent of the compiler [6].
- **Runtime safety checks:** re-check types, abilities and references with separate code, optionally asynchronously after commit [3].
- **MonoVM:** a new ultra fast VM engine with monomorphized code and flat value layouts, validated by replaying mainnet transactions against the existing VM [8].

5. Extending the platform to Rust

Sequential safe Rust and Move enforce the same aliasing rule: one mutable reference or many immutable ones, with bounded lifetimes. Rust programs can therefore be compiled to verified bytecode while keeping their safety guarantees checkable at the bytecode level. Compiling Rust to WebAssembly or native code discards these guarantees at the boundary. Beyond this safe core, the platform supports what real Rust code depends on: recursive data structures, interior mutability and unsafe raw pointers, each with its own checking discipline.

5.1 Source subset

The supported language is Rust that is:

- **Sequential:** no threads, atomics or `async` within a unit of execution. Parallelism comes from executing independent units concurrently, as Aptos does with Block-STM [7].
- **Deterministic:** no clock, OS randomness, environment or I/O, with deterministic replacements for randomized data structures.
- **Memory-checked:** safe code is checked by the borrow discipline, and `unsafe` code with raw pointers is checked by the memory model of Section 5.4.
- **IEEE 754 floating point under a deterministic profile:** fixed rounding mode, no fused or extended-precision shortcuts beyond specified operations, canonical NaN bit patterns, and a pinned, correctly rounded math library for transcendental functions.

Recursive types (lists, trees and graphs built with `Box` and `Rc`) and interior mutability (`Cell`, `RefCell`, `Rc`) are supported. Programs outside the subset are rejected with a diagnostic, not compiled approximately.

5.2 Compilation path

The frontend reads Rust MIR after borrow checking and monomorphization, through the Rustc Public (formerly Stable MIR) interface. MIR drops are explicit, `panic=abort` maps to abort, and generics are already resolved. The output is Move bytecode with four extensions:

1. **References inside structs.** Rust stores references in structs for iterators, slices and borrowed views. The bytecode format gains lifetime parameters on structs and functions, declared by the producer and checked by the verifier. Structs holding references cannot be stored globally, which confines them to stack lifetimes.
2. **Recursive types.** Move forbids recursive struct definitions. The format admits recursion through an owning indirection (`Box`, `Rc`), represented in MonoVM as a heap object, so flat layouts stay finite.
3. **Interior mutability.** `Cell`, `RefCell` and `Rc` map to native cells in the VM with identity, dynamic borrow state and reference counts; conflicting borrows abort.

4. **Raw pointers.** A pointer type and an instruction group for address-of, load, store, offset, allocation and deallocation, with the checked semantics of Section 5.4.

5.3 The verifier extension

The main technical work is sound, modular checking of lifetime-annotated bytecode from untrusted producers. The abstract domain of the reference-safety verifier generalizes from single references to values carrying one borrow identity per lifetime position. Calls are summarized precisely from declared lifetime signatures, replacing today's conservative summaries. Only checking is needed, not inference, so the analysis stays per-function and bounded, and its cost is metered.

The runtime reference checker extends in the same way, preserving the defense-in-depth property for Rust code.

5.4 Unsafe code under a checked memory model

In native Rust, misuse of a raw pointer is undefined behavior: the program loses all meaning, and the error is often silent. The platform gives every raw-pointer operation a checked semantics instead. Raw pointers are a separate domain from references: they carry no borrows, and aliasing is allowed only inside an explicit, checked memory.

- **Strict provenance.** A pointer is valid only within the allocation it was derived from; cross-allocation arithmetic is rejected.
- **Allocation identities are never reused.** A dangling pointer still names its dead allocation, so liveness can be checked at every access.
- **Typed allocations with a layout view.** An allocation holds typed values, possibly uninitialized. Typed accesses use element offsets; byte-level accesses go through the type's defined layout, and a load checks that its bytes form a valid value of the loaded type. Pointer bytes keep their provenance.

Each operation carries its safety conditions, and each misuse is detected at a specific site:

Misuse	Detected as
Use after free, owner drop or frame exit	Liveness failure at the access
Double free	Liveness failure at the deallocation
Out-of-bounds offset or access	Bounds failure at the offset or access
Read of uninitialized memory	Initialization failure at the load
Reinterpretation into an invalid value (e.g. a `bool` from byte 2), or a pointer from integer bytes with no exposed provenance	Validity failure at the load
Write through a null pointer or one derived from a shared reference	Permission failure at the store
Reference created from a raw pointer (`&*p`, `&mut *p`) that conflicts with a live reference to the same place, or is used after its allocation dies	Aliasing failure at the reborrow or the conflicting access; liveness failure at the use

With runtime checks enabled, the VM checks these conditions and aborts with a located error. The prover treats the same conditions as proof obligations, so it can show before deployment that no check fails.

Unsafe support exists so that existing Rust crates can be reused without rewriting, not to encourage new unsafe code. Runtime checking is the fallback for unproved code. Where the prover shows that no check in a function can fail, the deployer can let the VM skip those checks, as it already can for trusted code, so proved unsafe code keeps its performance.

An `unsafe fn` becomes a function whose contract has safety preconditions its callers must meet. Safe code and raw pointers to the same place share one memory cell and see each other's writes; a pointer derived from a live mutable reference must be finished with before the reference is used again.

The reverse direction is the hardest case. Turning a raw pointer back into a reference creates a reference with no statically checked origin, whose lifetime the code chooses freely. The checked semantics treats this as a dynamic reborrow: the new reference is registered with the runtime reference checker as a borrow of the target place and must obey the same rules as any other reference. A mutable reborrow requires that no other live reference to the place exists; while a shared reborrow lives, no write through any pointer may reach the place; and any use after the allocation dies is a liveness failure. The prover discharges the same conditions as proof obligations, which is how safe abstractions over unsafe code, such as a collection handing out references to its elements, are shown sound.

Pointers stored in data structures, as in linked lists and graphs, use a library type, `StoredPtr<T>`: a generational handle whose accesses are always checked. Its misuse is a defined abort, never undefined behavior, so data structures built from it need no memory-safety invariants.

Rust has no finished normative memory model. The platform follows the core on which the candidate models (Stacked Borrows, Tree Borrows [9, 10]) agree, and covers the full set of unsafe operations: unions, transmutes and byte-level reinterpretation through the layout view, and integer-to-pointer casts through exposed provenance. The prover covers the full set first, and the VM's runtime checks follow (Section 5.5). Inline assembly and FFI leave the Rust semantics and are not supported.

5.5 Staging

Stage	Scope	Outcome
0	Rust MIR to current bytecode; references in structs lowered to arenas and indices; native cells for `RefCell`	Working pipeline and a measured coverage of real Rust code, with no format changes
1	Lifetime annotations and recursive types in the format and runtime support, verifier bypassed in trusted configurations	Genuine reference fields for experimentation
2	Lifetime-checked verifier and runtime checks for untrusted bytecode	Full guarantees for Rust code from any producer
U	Raw pointers under the checked memory model, first in the Lean-based verifier, then in the VM's runtime checks; unions, reinterpretation and integer-pointer casts through the layout and provenance model; `Box`-class and `StoredPtr` library models	Unsafe Rust with located runtime checks and provable absence of misuse

For safe Rust, we estimate the runtime at about 10% of the effort, the MIR frontend at 20 to 30%, and the verifier at 60 to 70%. Unsafe-pointer support is a separate track that runs alongside Stages 1 and 2.

6. Verification for Rust and numeric code

Verification follows two complementary paths: fast automated checking for daily development, and kernel-checked proofs where certification requires them.

6.1 Automated verification

The Move Prover's methods carry over to Rust because they rest on the shared reference discipline. Reference elimination gives an alias-free model without separation logic, and behavioral predicates handle closures and trait objects that modify state [2]. Specifications are written as contracts on functions and invariants on types, and conditions are discharged by SMT solvers in CI.

6.2 Kernel-checked verification

Our Lean-based verifier uses a typed intermediate representation in Lean 4 with semantic profiles for Move and Rust. Frontends import borrow-checked MIR and Move bytecode into the same representation. A `verify` command proves a contract as a theorem about the program's semantics, checked by the Lean kernel. Differential execution against MonoVM tests that both sides agree on program meaning. A theorem connecting the Lean semantics to the compiled bytecode is future work.

Its semantics includes the checked raw-pointer model of Section 5.4. Absence of undefined behavior is part of every proved contract, and each misuse appears as a failed proof obligation at a named location.

6.3 Specification inference

Writing specifications is the main cost of verification. Weakest-precondition analysis already infers complete abort and postconditions for loop-free Move code, including higher-order code, and inferred specifications feed back into the prover as an independent check [4]. Combined with AI-based inference of loop invariants and intent-level properties, this changes the economics: people review and approve specifications instead of writing them.

6.4 Numeric code

SMT floating-point theories scale poorly for real programs. We instead specify numeric functions over real numbers with explicit error bounds, for example that a filter's output is within a stated distance of its exact real-valued result. Bounds are discharged by interval and error analysis, with Lean proofs for the cases automation cannot close. Deterministic floating point (Section 5.1) makes these results reproducible on every platform.

7. Evidence to date

The platform's claims rest on production use, not prototypes.

Area	Evidence
Prover in production	Runs in CI on the Aptos framework, with specifications of its critical modules, including the block prologue, staking, metering and code deployment [5]
Higher-order verification	About 240 verified functions and 66 expected-error cases in the closure test suite, stable across solver versions, under 5 minutes on standard CI runners [2]
Runtime checks	Proof-of-concept exploits from bug-bounty reports and audits passed the static verifier and were stopped by the runtime checks [3]
Runtime check cost	Type and ability checks cost about 10% of throughput in asynchronous mode at production concurrency; the reference checker costs 26 to 29% before optimization and is not yet in production
No false positives	Reference checker re-executed billions of historical Aptos transactions without a false alarm

MonoVM's throughput, measured on recorded blocks of realistic workloads, is 30 to 50 times that of the current Move VM.

These comparisons are against the current Move VM. Comparisons with WebAssembly runtimes and native Rust on the same workloads are in preparation.

8. Application domains

The platform fits code that must be correct, must run in isolation, and benefits from deterministic replay.

8.1 Execution of agent-generated code

AI agents increasingly write and run programs: tools, data transformations, workflows and policy logic. The platform runs such code in a metered, isolated sandbox whose safety does not depend on the code's author. Properties can be inferred and proved before execution, for example that a program touches only declared resources, never exceeds a budget, or preserves an account invariant. Because agents write ordinary Rust, including recursive data structures, shared mutable state and occasional `unsafe` code, the platform accepts that code as written instead of forcing a restricted style. Deterministic execution makes every run replayable for audit.

8.2 Financial logic

Settlement, clearing, margin and risk-limit rules, pricing functions and order matching are small, high-value programs where errors are expensive and auditors ask for reproducibility. The platform's origin in on-chain finance means these workloads are already its best-tested case. Resource types model assets that cannot be duplicated or lost, and global invariants express conservation rules directly.

8.3 Safety-critical and defense systems

Control laws, filters, guidance and mission logic need proved properties, bounded resource use and reproducible numerics. With deterministic floating point and error-bound specifications, the platform addresses these needs. Hard real-time loops typically run as native code verified with the same tools, with MonoVM used for simulation and shadow execution. In avionics terms, the design is dissimilar redundancy applied to software: the prover, the verifier, the runtime checks and the MonoVM shadow channel are independently implemented mechanisms that cross-check each other. Certification under standards such as DO-178C also requires qualified tools and long-term support; we treat this as a later phase built on the same technology.

8.4 Multi-party computation of trusted logic

When one organization runs code supplied by another (a partner's pricing model, a regulator's rule, a customer's extension), the platform gives the host guarantees without reviewing or trusting the supplier's toolchain.

8.5 Checked execution of existing Rust code

Most production Rust depends on `unsafe` somewhere: in collections, parsers, codecs and performance-critical libraries. Because raw pointers, recursive types and interior mutability are supported, such code can run on the platform without being rewritten into a restricted style, within the limits of Section 5.1. Memory errors that would silently corrupt state natively become defined, located aborts, and the prover can establish their absence for components that warrant it. This makes the platform a hardened runtime for plugins, extensions and third-party libraries, and a way to audit unsafe code with proofs rather than by review alone.

9. Relation to existing approaches

No existing system combines a verifier, independent load-time and runtime safety checks, and deterministic metered execution for one language.

Approach	Proves functional properties	Checks untrusted code at load	Independent runtime checks	Ownership and resources at runtime	Deterministic, metered	Execution performance
Verex	Yes (SMT and Lean)	Yes	Yes	Yes	Yes	Optimized interpreter with flat,

Approach	Proves functional properties	Checks untrusted code at load	Independent runtime checks	Ownership and resources at runtime	Deterministic, metered	Execution performance
						monomorphized values (30–50× the previous Move VM); native compilation of verified Rust where interpretation is too slow; ahead-of-time compilation planned
WebAssembly runtimes	No	Structural validation only	Memory bounds	No	Deterministic profile, metering by embedder	Near native (JIT or AOT compilation)
JVM, CLR	No	Yes	Partial (casts, array stores, bounds)	No	No	Near native (JIT compilation)
Rust verifiers (Verus, Kani, Creusot, Prusti, Aeneas)	Yes	No	No	No	No	Native (verification only; no runtime of their own)
SPARK / Ada	Yes	No	Optional runtime checks	No	No	Native
K framework (Runtime Verification)	Yes	No	Only as defined by the language semantics	Only as defined by the language semantics	Per language semantics	Generic semantics interpreter; slower than dedicated engines
Miri (Rust UB checker)	No	No	Yes, as a testing tool	Aliasing only, no resources	No	Two or more orders of magnitude slower than native
EVM, Solana VM	No	No	No	No	Yes	EVM interpreted; Solana JIT-compiled
Sui Move	Yes (Move Prover)	Yes	No	No	Yes	Interpreted (Move VM)
Linux eBPF verifier	No	Yes (memory safety, termination)	No	No	Bounded by the verifier	Near native (JIT compilation)
CHERI	No	No	Yes, in hardware	No	No	Native, on CHERI

Approach	Proves functional properties	Checks untrusted code at load	Independent runtime checks	Ownership and resources at runtime	Deterministic, metered	Execution performance
			(bounds and provenance)			processors
Zero-knowledge VMs (RISC Zero, SP1, Jolt, Cairo)	No (prove that execution happened, not that the program is correct)	No	No	No	Yes	Proving several orders of magnitude slower than native; verification fast

The closest work on runtime checking for Rust is Miri with the Stacked Borrows and Tree Borrows aliasing models [9, 10]. It aims to define Rust's full unsafe semantics for testing and tolerates overheads of two or more orders of magnitude. Our runtime checks are designed for production. For safe code, the reference checker never rejects a program that a correct verifier accepts. For unsafe code, our provenance-based model is stricter than Miri's and can reject programs Miri accepts, in exchange for checks cheap enough for production.

Other systems check untrusted code at a single layer. The Linux eBPF verifier [11] proves memory safety and termination of kernel extensions at load time, with no independent runtime check behind it. CHERI [12] enforces bounds and provenance in hardware, but has no notion of ownership or resources and needs special processors. Sui uses the same Move language and bytecode verifier, but without independent runtime checks or MonoVM.

Zero-knowledge VMs answer a different question: they prove to a third party that a given program ran on given inputs and produced a given output, without re-executing it. They say nothing about whether the program is safe or correct; a buggy or malicious program yields a valid proof of its faulty run. Most compile Rust to RISC-V, so they also trust the compiler and inherit undefined behavior from `unsafe` code. The two approaches compose: a program verified and checked on our platform is a sound guest for a zkVM.

The K framework takes the generic route: from a formal semantics of any language (EVM, WebAssembly, C and others) it derives an interpreter, symbolic execution and a deductive verifier, so one definition serves both execution and proof. We take the specialized route and design every layer for the Move and Rust ownership discipline. The verifier, the runtime checks and the prover all rely on this discipline: it makes bytecode checkable without trusting the compiler, gives the prover an alias-free memory model with simple SMT encodings, and lets MonoVM use flat, statically laid-out values. A generic framework cannot rely on properties that are specific to one language, and pays for its generality in execution speed and in proof automation.

Among verifiers, Dafny, F* and Verus support higher-order functions. Dafny does not allow function values to modify state, and F* and Verus reason about memory through separation logic or state threading. Move's static memory separation lets the prover handle stateful higher-order code with simpler SMT encodings.

10. Roadmap and open problems

10.1 Roadmap

Phase	Horizon	Deliverables
1	0–12 months	Rust Stage 0 pipeline with native cells for interior mutability; deterministic floating-point profile; checked raw-pointer model in the Lean-based verifier; MonoVM as standalone runtime outside blockchain; benchmarks against WebAssembly and native Rust; first design-partner deployments

Phase	Horizon	Deliverables
2	12–24 months	Lifetime-annotated bytecode and recursive types (Stage 1); checked raw pointers in the VM; prover support for the Rust subset; production reference checker with waiver and asynchronous modes; AI-assisted specification inference as a product
3	24–36 months	Lifetime-checked verifier for untrusted Rust bytecode (Stage 2); `Box`-class and `StoredPtr` library models verified against the memory model; Lean path with compiler-correctness results for core fragments; ahead-of-time compilation from verified bytecode; groundwork for tool qualification

10.2 Open problems

- **Sound lifetime checking of adversarial bytecode.** The central research question; needs a soundness argument extending the formalized Move borrow checker [6].
- **Precision against metering.** Lifetime positions increase the size of the borrow graph, and verification cost must stay bounded for untrusted input.
- **Ecosystem coverage.** Much real Rust relies on threads, FFI and crates outside the subset. Stage 0 will measure how much code fits.
- **Fidelity for unsafe code.** Without a normative Rust memory model, the checked model must stay conservative enough to be sound and permissive enough to run real libraries. The rules for turning raw pointers back into references are where the candidate models differ most.
- **Cost of checked raw accesses.** Pointer-heavy inner loops pay for liveness and bounds checks; how many of them proofs can remove is open.
- **Performance against native code.** An interpreter will be significantly slower than native execution, and some applications cannot afford it. They can run verified Rust natively, with MonoVM for simulation, shadow execution and differential testing, at the cost of the load-time and runtime checks. JIT or ahead-of-time compilation from verified bytecode is the planned answer for keeping all layers at near-native speed and must preserve the runtime checks. With flat, monomorphized values and statically known layouts, a compiled MonoVM can be expected to perform similarly to WebAssembly runtimes, which run compute-heavy code at about 1.5 to 2.5 times native time [13].
- **Specification effort.** Inference reduces but does not remove the need for people to state what code should do; loop invariants and numeric error bounds remain the hardest cases.
- **Certification.** Standards for safety-critical software require tool qualification and long-term support processes beyond the technology.

References

1. D. Dill, W. Grieskamp, J. Park, S. Qadeer, M. Xu, E. Zhong. Fast and Reliable Formal Verification of Smart Contracts with the Move Prover. TACAS 2022.
2. W. Grieskamp, T. Zhang, V. Kashyap, J. Silverman. Formal Verification of Imperative First-Class Functions in Move. 2026.
3. V. Gao, W. Grieskamp, V. Kashyap, G. Mitenkov, T. Zhang, R. Zhou, A. Cappa, M. Ilardi. Defense-in-Depth Runtime Safety in Move. 2026.
4. W. Grieskamp, T. Zhang, V. Kashyap. Combining Mechanical and Agentic Specification Inference for Move. 2026.

5. J. Park, T. Zhang, W. Grieskamp, M. Xu, G. Di Giacomo, K. Chen, Y. Lu, R. Chen. Securing Aptos Framework with Formal Verification. FMBC 2024.
6. S. Blackshear, J. Mitchell, T. Nowacki, S. Qadeer. The Move Borrow Checker. 2022.
7. R. Gelashvili et al. Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing. PPoPP 2023.
8. Aptos Labs. aptos-core, third_party/move: MonoVM design documents and benchmarks (`mono-move/`), RustMV design note (`brainstorming/rustmv.md`), the Lean-based verifier and its unsafe-pointer design (`lean/`, `lean/designs/unsafe-pointers.md`).
9. R. Jung, H.-H. Dang, J. Kang, D. Dreyer. Stacked Borrows: An Aliasing Model for Rust. POPL 2020.
10. N. Villani, J. Hostert, D. Dreyer, R. Jung. Tree Borrows. PLDI 2025.
11. E. Gershuni et al. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. PLDI 2019.
12. R. Watson et al. CHERI: A Hybrid Capability-System Architecture for Scalable Software Compartmentalization. IEEE S&P 2015.
13. A. Jangda, B. Powers, E. Berger, A. Guha. Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code. USENIX ATC 2019.

Appendix A. Company framing (internal)

Internal working notes, not part of the technical whitepaper. Remove before external distribution.

A.1 Concept

A startup spun out of Aptos Labs, with Aptos support, that builds Verex on the Move platform (prover, bytecode verifier, runtime safety checks, MonoVM) and extends it to Rust as described in Sections 5 and 6. One line: run code you did not write and cannot fully review, with guarantees that do not depend on any single tool being correct. Why now: AI writes much of the code, and AI-assisted auditing finds bugs faster.

A.2 Markets and wedge

1. **Lead: execution of agent-generated code and actions.** A metered, isolated, replayable sandbox with proved or inferred properties.
2. **Financial logic.** Settlement, clearing, risk limits and pricing; floating point extends this to risk and pricing models.
3. **Multi-party code execution.** Running a partner's or regulator's code inside one's own system.
4. **Checked execution of existing Rust.** Hardened runtime for plugins and third-party libraries with unsafe code (Section 8.5).
5. **Later phase: safety-critical, defense, space and avionics.** Needs DO-178C tool qualification and long timelines.

Not claimed: large-model inference, which belongs on GPU/SIMD. Open decision: lead wedge, agent-code execution or financial logic.

A.3 Competition and positioning

- **Wasm runtimes:** a sandbox without ownership or resource semantics.
- **Rust verifiers (Verus, Kani, Creusot, Prusti, Aeneas):** prove code but provide no runtime.

- **SPARK/Ada:** the incumbent in avionics and defense.
- **Certora:** the blockchain verification comparable.
- **Zero-knowledge VMs (RISC Zero, Succinct SP1, a16z Jolt, StarkWare Cairo):** well funded and pitched as "verifiable execution", but they prove that code ran, not that it is correct. Position as complementary, not competing.
- **Runtime Verification (K framework):** generic semantics framework that derives execution, symbolic execution and verification from one language definition (KEVM, KWasm, C). Mostly a blockchain audit and verification services business. The related Pi Squared (also founded by Grigore Roșu) pitches verifiable execution.
- **Specialized, not generic:** our focus on Move and Rust is a strength. A system built around one ownership discipline checks, proves and executes better than a framework that must accept any language (Section 9).
- **JVM/CLR:** load-time verification only.

Answer to "why not Wasm plus Verus?": Move bytecode keeps Rust's ownership discipline and re-checks it at load and at run time, with one integrated path from specification to execution.

A.4 Aptos relationship and structure

- **Aptos support** resolves IP and team transfer. Aptos can be the first customer, for verification services and maintenance.
- **Cap table and governance:** keep Aptos a minority shareholder, with a broad, exclusive non-blockchain license.
- **Risks:** shared-code maintenance and fork risk; crypto perception (use a separate brand); revenue concentration on Aptos.
- **To prepare:** a structure slide covering the license, the Aptos stake, the customer relationship and the shared-code arrangement.

A.5 Founding team

The founding team is constituted from the Aptos Move and Security teams.

A.6 Assessment and odds

Judgment from the discussion, not data.

- **Strengths:** a team with shipped production systems; timing (AI plus verification); a distinctive "no single trusted component" story.
- **Weaknesses:** adoption cost of a new runtime; formal-methods companies historically stay small; finding users is the core problem; the Rust verifier carries research risk.

Outcome	Odds
Seed round	60–75%
Paying outside design partners within 18 months	30–40%
Large durable company	Under 10%

Levers that most improve the odds: a commercial co-founder; 2–3 design partners before raising; Aptos as partner or shareholder; early services revenue; one narrow wedge; benchmarks against Wasmtime and native Rust before the first investor meeting.

A.7 Next steps

- Benchmarks against Wasmtime and native Rust on 2–3 MonoVM workloads.
- Pitch deck and spin-out structure slide.
- Decide the lead wedge.
- Identify and approach design partners.